End-to-End CI/CD Pipeline for a Calculator REST API

CSE 816 Software Production Engineering Mini-Project

Student: Harsh Gupta **Roll number:** IMT2023121

GitHub repository	https://github.com/Reverent2005/calculator-devops
Docker Hub image	https://hub.docker.com/r/reverent/calculator-api

1. Overview and architecture

This project implements a production-style Calculator REST API using Spring Boot, Maven, Docker, Docker Compose, Jenkins, GitHub, and Docker Hub. The API supports addition, subtraction, multiplication, division, structured validation errors, and an Actuator health endpoint. Nginx acts as the public reverse proxy while the API remains on the internal Docker network.

→	**Jenkins**	→	**Checkout → Build → Test → Docker Build → Push → Deploy**	→

2. Pipeline and engineering decisions

The declarative Jenkinsfile checks out the triggering revision, compiles and packages with Maven, runs tests and publishes JUnit XML results, builds a multi-stage Docker image, pushes build-number and latest tags to Docker Hub using Jenkins-managed credentials, and deploys the immutable build tag with Docker Compose. The deployment is idempotent and verifies the proxied Actuator health endpoint. The Dockerfile separates the Maven build image from a slim non-root JRE runtime. Compose adds health checks, restart policies, environment-driven configuration, and an Nginx service.

3. Testing and outcome

Seven automated tests passed: five service tests cover all arithmetic operations and division by zero, and two Spring MVC integration tests verify a successful endpoint response and a structured HTTP 400 error. The evidence below shows the successful webhook, Jenkins pipeline, test report, notification, running containers, and API behavior.

4. CI/CD evidence

Jenkins pipeline



Figure 1. Build #3 completed with green Checkout, Build, Test, Docker Build, Push, and Deploy stages.

GitHub webhook

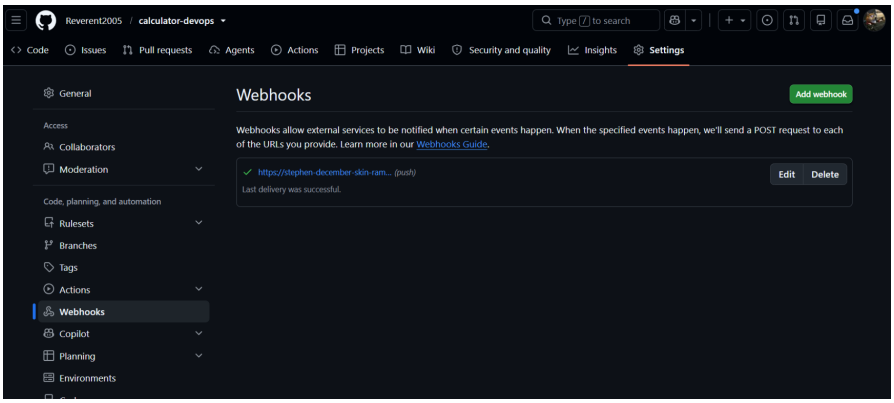


Figure 2. GitHub reports the webhook delivery as successful.

5. Verification evidence

JUnit report

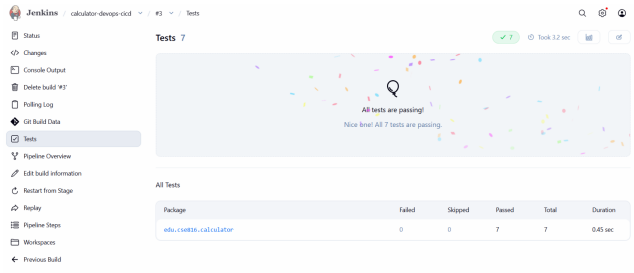


Figure 3. Jenkins reports all 7 tests passed.

Build notification

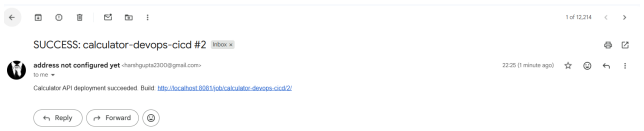


Figure 4. Jenkins success email for build #2.

Compose deployment

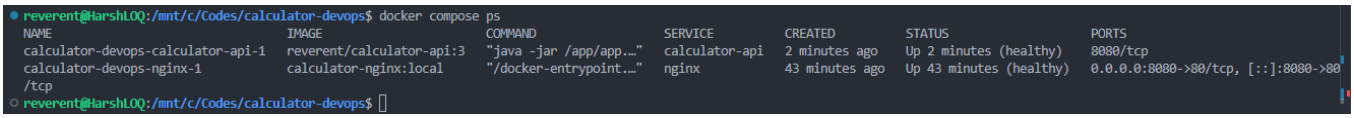


Figure 5. Calculator API and Nginx containers are healthy.

API behavior

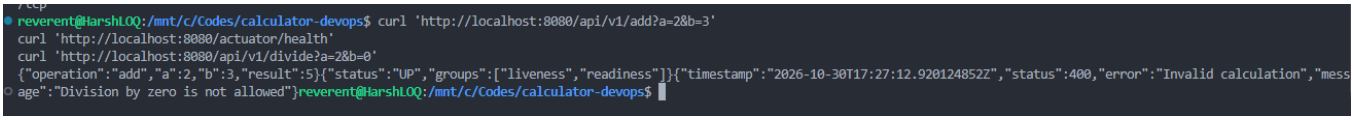


Figure 6. API addition, health UP, and division-by-zero HTTP 400 responses.

6. Challenges and conclusion

The main integration challenge was connecting a locally hosted Jenkins instance to GitHub. A temporary Cloudflare Quick Tunnel provided a public HTTPS endpoint for webhook delivery; it is started only for the demonstration and stopped afterward. Other challenges included deploying the exact Jenkins build image and securing Docker Hub authentication. These were addressed with build-number image tags, Compose environment variables, health polling, and Jenkins credential binding. The completed pipeline provides a repeatable workflow from a GitHub push through automated testing, container publishing, deployment, and email notification.